

Case Machine Learning Engineer



O **PicPay** é uma das maiores fintechs do Brasil, oferecendo soluções digitais de pagamentos, carteira, crédito e serviços financeiros para milhões de usuários. Nossa missão é simplificar a vida financeira das pessoas por meio de tecnologia, dados e produtos intuitivos.

Somos uma empresa orientada por dados. Trabalhamos com sistemas distribuídos, modelos de machine learning em produção e serviços de alta escala, sempre com foco em confiabilidade, segurança e experiência do usuário.

Este case foi elaborado para refletir desafios reais enfrentados pelas equipes de Engenharia de Dados e Machine Learning do PicPay, avaliando tanto a capacidade técnica quanto a maturidade em engenharia e organização de soluções.

Escopo do Case

Este case técnico tem como objetivo avaliar competências essenciais para a posição de Engenheiro(a) de Machine Learning no PicPay.

O desafio é dividido em duas partes complementares:

1. **Análise de dados a partir de uma API pública**, envolvendo ingestão, modelagem e processamento com Spark.
2. **Desenvolvimento de um microserviço de inferência online**, utilizando um modelo de NER para extração estruturada de informações a partir de texto.

O foco da avaliação está na capacidade de construir soluções organizadas, reproduzíveis e próximas de um ambiente real de produção, aplicando boas práticas de engenharia de software, dados e ML.

Análise de Dados com PokeAPI

Objetivo

Nesta etapa, o(a) candidato(a) deverá consumir dados diretamente da API pública da PokeAPI e realizar análises utilizando Apache Spark.

O objetivo é avaliar habilidades de:

- Ingestão de dados a partir de API REST
 - Tratamento de paginação e dados aninhados (JSON)
 - Modelagem relacional
 - Manipulação e agregação de dados com Spark
 - Clareza na organização e documentação do código
-

Fonte de Dados

A API a ser utilizada é:

<https://pokeapi.co/api/v2/pokemon>

Veja a doc em: <https://pokeapi.co/docs/v2#pokemon-section>

Etapas

Etapa 1 — Extração de Dados

Consumir o endpoint `/pokemon` para obter a listagem completa de todos os pokémons.

Exemplo de retorno:

```
{  
  "name": "bulbasaur",  
  "url": "https://pokeapi.co/api/v2/pokemon/1/"  
}
```

1. Para cada `url` retornada, realizar uma nova requisição e extrair exclusivamente os seguintes campos e transformá los em novas tabelas:
 - `types`
 - `stats`
 - `abilities`
-

Etapa 2 — Modelagem dos Dados

Ao final da extração, os dados deverão estar organizados de acordo com as tabelas apresentadas no tópico [Dicionário de Dados](#) na seção [Anexos](#) no final desse documento.

Etapa 3 — Análises com Spark

Após a construção das tabelas, o(a) candidato(a) deverá responder às seguintes perguntas utilizando Apache **Spark**:

Caso você não tenha um ambiente Spark, tente ver o [Databricks Community Edition](#)

1. Pokémons com múltiplos tipos e força acima da média

Definição de força: soma de todos os `base_stat` de cada pokémon.

Pergunta:

Quantos pokémons possuem mais de um `type_name` e têm a força maior que a média geral de todos os pokémons?

2. Abilities exclusivas de pokémons com múltiplos tipos

Pergunta:

Quais habilidades não aparecem em nenhum pokémon de tipo único?

Queremos identificar aquelas habilidades que só existem em pokémons que têm mais de um tipo.

Exemplo :

Habilidade levitate → só aparece em pokémons com 2 tipos 

Habilidade overgrow → aparece em pokémons com 1 tipo 

3. Pokémons mais versáteis

Pergunta: Quais são os 5 pokémons que apresentam maior versatilidade de acordo com este critério?

A métrica de cálculo estabelecida para o índice de versatilidade é a seguinte:

`versatility_score = (número de tipos * 2) + (número de abilities) + (soma dos stats / 100)`

Exemplo:

- Pokémon X → Pontuação = 14.2
- Pokémon Y → Pontuação = 1

Desenvolvimento do Micro Serviço de Model Serving

Objetivo

O PicPay possui um serviço de PIX por WhatsApp onde o cliente envia um texto ou áudio para o assistente de pagamentos do PicPay no WhatsApp solicitando uma operação bancária da seguinte forma:

Faça um PIX de R\$10,00 para o João amanhã

O assistente então interpreta essa mensagem e realiza a operação financeira solicitada.

O objetivo dessa etapa é construir um micro serviço que receba uma mensagem desse tipo e retorne os dados que serão utilizados para dar continuidade na transação, ex:

- **OPERACAO:** PIX
- **VALOR:** 10,00
- **DESTINO:** João
- **DATA:** amanhã

O serviço deve ser construído em **Python**, containerizado utilizando **Docker** e executável localmente.

Requisitos mínimos da API

O micro serviço deve conter, no mínimo, os seguintes endpoints:

POST /load/

Registra uma nova versão do modelo.

- **Payload:**
 - **model:** str - Nome do modelo spaCy a ser utilizado, ex: `en_core_web_sm`, `en_core_web_md`, `en_core_web_lg`
 - **Função:** Ao passar o nome do modelo spaCy o ms deverá ver se já tem esse modelo baixado, caso contrário baixar o modelo e deixar a disposição,
 - **Dica:** Ex de download `python3 -m spacy download en_core_web_sm`
-

POST /predict/

Realiza inferência utilizando o modelo ativo ou uma versão específica.

- **Payload:**
 - **text:** str - Texto (em ingles) a ser analisado
 - **model:** str - Nome do modelo spaCy a ser utilizado, ex: `en_core_web_sm`, `en_core_web_md`, `en_core_web_lg`
- **Função:** Carrega o modelo solicitado e realiza inferência utilizando o modelo passado como parâmetro

- **Exemplo de Entrada:** *Can you send \$45 to Michael on June 3?*
 - **Exemplo de Saída:**
 - **MONEY** : 45
 - **PERSON** : Michael
 - **DATE** : June 3
 - **Dica:** Ex de load do modelo: `spacy.load("en_core_web_sm")`
-

GET /list/

Lista as predições realizadas.

- Deve retornar histórico de:
 - Input recebido
 - Output gerado
 - Timestamp
 - Versão do modelo utilizada
-

Endpoints adicionais (sugeridos)

O candidato pode propor e implementar outros endpoints que considere relevantes. Exemplos:

- **GET /models/** → lista versões disponíveis
 - **GET /health/** → healthcheck do serviço
 - **DELETE /models/{version}** → remove versão
-

Requisitos técnicos

- Linguagem: **Python**
 - Framework sugerido: FastAPI, Flask ou similar
 - Containerização obrigatória com **Docker**
 - Serviço deve rodar localmente
-

Boas práticas esperadas

- Organização de código (estrutura de projeto)
- Separação de responsabilidades
- Tratamento de erros
- Logs estruturados
- Versionamento claro
- README explicando como rodar

Formato de Submissão do Desafio

A submissão do desafio deverá ser efetuada por meio de um repositório Git (GitHub, GitLab ou Bitbucket).

Requisitos de Submissão

- O repositório deve ser público, assegurando o acesso para fins de avaliação.
- O link do repositório deve ser encaminhado conforme as diretrizes do processo seletivo.
- A totalidade do código necessária para a execução e avaliação deve estar versionada no repositório.

Conteúdo Mínimo Requerido no Repositório

O repositório deverá conter, minimamente:

1. Parte 1 — Análise de Dados

- Um arquivo `notebook.py` ou *notebook* (`.ipynb`) compreendendo:
 - Extração dos dados da *PokeAPI*.
 - Construção das tabelas especificadas.
 - Resolução das análises propostas com a utilização do *Spark*.

2. Parte 2 — Microserviço

- Código-fonte do microserviço de análise de texto (*NER*).
- Implementação da API em conformidade com o que foi detalhado no enunciado.
- Arquivos essenciais para a execução do serviço (e.g., `Dockerfile`, `requirements.txt` ou equivalente).

3. Documentação

- Um arquivo `README.md` localizado na raiz do repositório explicando a estrutura do projeto.

Anexos

Dicionário de Dados

 Tabela: **pokemon**

Coluna	Tipo	Descrição
pokemon_id	int	Identificador único do pokémon na PokeAPI. Chave primária lógica.
name	string	Nome do pokémon (encontrado na url <code>https://pokeapi.co/api/v2/ability/<id>/</code>).
height	int	Altura do pokémon em decímetros (encontrado na url <code>https://pokeapi.co/api/v2/ability/<id>/</code>).
weight	int	Peso do pokémon em hectogramas(encontrado na url <code>https://pokeapi.co/api/v2/ability/<id>/</code>).
base_experience	int	Experiência base concedida ao derrotar o pokémon.(encontrado na url <code>https://pokeapi.co/api/v2/ability/<id>/</code>).

 Tabela: **pokemon_type**

Coluna	Tipo	Descrição
pokemon_id	int	Identificador do pokémon. Chave estrangeira para pokemon .
type_name	string	Nome do tipo do pokémon (ex: fire, water, grass, flying).

Observação:

A combinação (**pokemon_id**, **type_name**) representa a chave composta da tabela.

 Tabela: **pokemon_stats**

Coluna	Tipo	Descrição
<code>pokemon_id</code>	int	Identificador do pokémon. Chave estrangeira para <code>pokemon</code> .
<code>stat_name</code>	string	Nome do stat (hp, attack, defense, special-attack, special-defense, speed).
<code>base_stat</code>	int	Valor base daquele stat para o pokémon.

Observação:

Cada pokémon possui múltiplos registros nesta tabela, um para cada stat.

 Tabela: **pokemon_ability**

Coluna	Tipo	Descrição
<code>pokemon_id</code>	int	Identificador do pokémon. Chave estrangeira para <code>pokemon</code> .
<code>ability_name</code>	string	Nome da habilidade associada ao pokémon.
<code>is_hidden</code>	boolean	Indica se a habilidade é oculta (<code>true</code>) ou comum (<code>false</code>).
